

LAB 1

Implementation and Performance Analysis of Q-Learning Using an Epsilon-Greedy Policy

This lab uses Gymnasium's deterministic `CliffWalking-v1` environment instead of an Atari environment. Its discrete state/action spaces make a tabular Q-learning implementation easy to inspect.

Aim and tasks

Train a Q-learning agent with an epsilon-greedy policy using **500, 1,000, 2,000, 5,000, and 10,000 episodes**. Epsilon starts high and decays during training. The notebook records every episode's reward, plots cumulative and latest-100 average reward, evaluates each learned policy for 100 greedy episodes, compares success rates, and visualizes the environment and extracted policy.

```
In [1]: import gymnasium as gym
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

EPISODE_CONFIGS = [500, 1_000, 2_000, 5_000, 10_000]
TEST_EPISODES = 100
MAX_STEPS = 100
SEED = 42
LEARNING_RATE = 0.8
DISCOUNT_FACTOR = 0.95
EPSILON_START = 1.0
EPSILON_MIN = 0.01
EPSILON_DECAY = 0.001
```

```
In [2]: def train_q_learning(number_of_episodes, seed=SEED):
    env = gym.make("CliffWalking-v1")
    rng = np.random.default_rng(seed)
    q_table = np.zeros((env.observation_space.n, env.action_space.n))
    rewards, epsilon = [], EPSILON_START

    for episode in range(number_of_episodes):
        state, _ = env.reset(seed=seed + episode)
        total_reward = 0.0
        for _ in range(MAX_STEPS):
            if rng.random() < epsilon:
                action = int(rng.integers(env.action_space.n))
            else:
                action = int(np.argmax(q_table[state]))
            next_state, reward, terminated, truncated, _ = env.step(action)
```

```

        target = reward if terminated else reward + DISCOUNT_FACTOR * np.max(q_
        q_table[state, action] += LEARNING_RATE * (target - q_table[state, acti
        state, total_reward = next_state, total_reward + reward
        if terminated or truncated:
            break
        rewards.append(total_reward)
        epsilon = max(EPSILON_MIN, epsilon * np.exp(-EPSILON_DECAY))
    env.close()
    return q_table, np.asarray(rewards)

def evaluate_policy(q_table, number_of_episodes=TEST_EPISODES, seed=SEED):
    env = gym.make("CliffWalking-v1")
    successes = 0
    for episode in range(number_of_episodes):
        state, _ = env.reset(seed=seed + episode)
        for _ in range(MAX_STEPS):
            state, _, terminated, truncated, _ = env.step(int(np.argmax(q_table[sta
            if terminated:
                successes += 1
            if terminated or truncated:
                break
    env.close()
    return 100 * successes / number_of_episodes

```

```

In [3]: results, summary = {}, []
for episodes in EPISODE_CONFIGS:
    q_table, rewards = train_q_learning(episodes)
    results[episodes] = {"q_table": q_table, "rewards": rewards}
    summary.append({
        "Training episodes": episodes,
        "Final cumulative reward": rewards.sum(),
        "Average reward (last 100)": rewards[-100:].mean(),
        "Test success rate (%)": evaluate_policy(q_table),
    })
summary_df = pd.DataFrame(summary)
summary_df

```

```

Out[3]:

```

	Training episodes	Final cumulative reward	Average reward (last 100)	Test success rate (%)
0	500	-373334.0	-432.41	100.0
1	1000	-495121.0	-170.93	100.0
2	2000	-594297.0	-61.61	100.0
3	5000	-673884.0	-17.32	100.0
4	10000	-753359.0	-17.37	100.0

```

In [4]: fig, axes = plt.subplots(1, 2, figsize=(15, 5))
for episodes, result in results.items():
    rewards = result["rewards"]
    axes[0].plot(np.cumsum(rewards), label=f"{episodes:,} episodes")

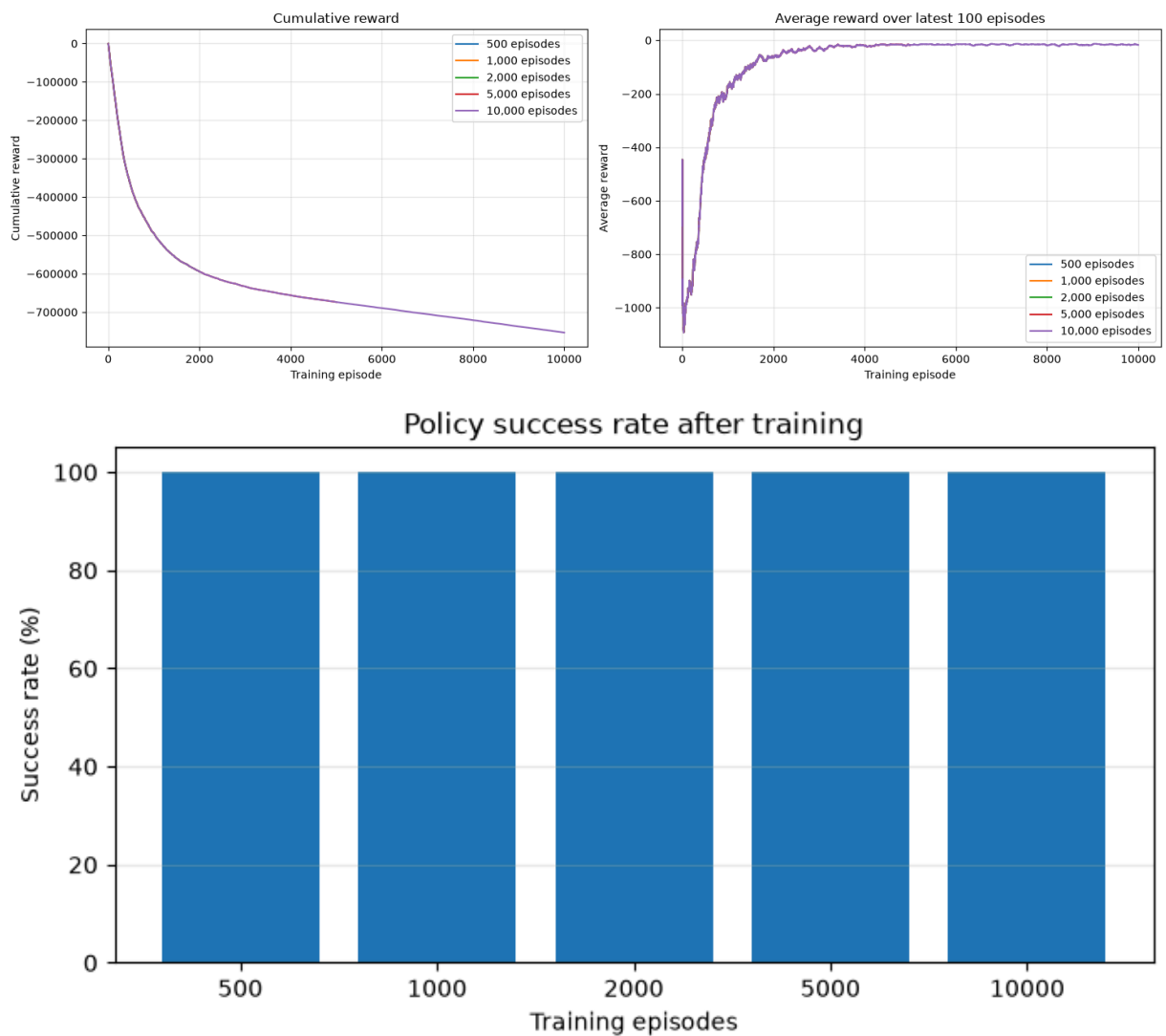
```

```

    axes[1].plot(pd.Series(rewards).rolling(100, min_periods=1).mean(), label=f"{ep
axes[0].set(title="Cumulative reward", xlabel="Training episode", ylabel="Cumulative
axes[1].set(title="Average reward over latest 100 episodes", xlabel="Training episo
for ax in axes:
    ax.grid(alpha=0.3)
    ax.legend()
plt.tight_layout()
plt.show()

plt.figure(figsize=(8, 4))
plt.bar(summary_df["Training episodes"].astype(str), summary_df["Test success rate
plt.title("Policy success rate after training")
plt.xlabel("Training episodes")
plt.ylabel("Success rate (%)")
plt.ylim(0, 105)
plt.grid(axis="y", alpha=0.3)
plt.show()

```



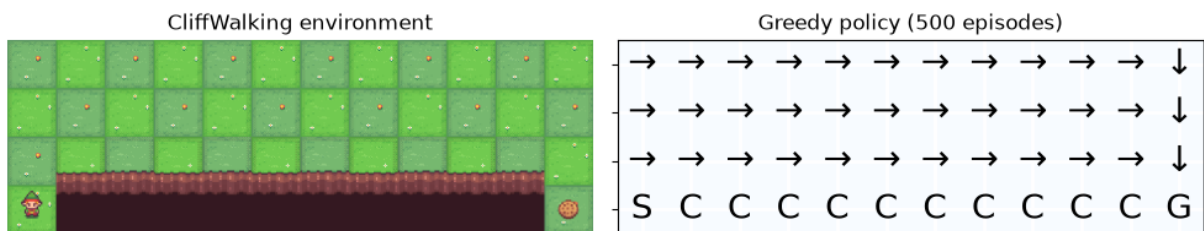
Environment and extracted policy

The arrows show the greedy action selected by the best-performing Q-table. **S** is the start, **C** marks the cliff, and **G** is the goal.

```
In [5]: best_episodes = int(summary_df.loc[summary_df["Test success rate (%)"].idxmax(), "T
policy = np.argmax(results[best_episodes]["q_table"], axis=1).reshape(4, 12)
symbols = {0: "↑", 1: "→", 2: "↓", 3: "←"}
cliff = np.array([list(row) for row in ["FFFFFFFFFFFF", "FFFFFFFFFFFF", "FFFFFFFFFF

fig, axes = plt.subplots(1, 2, figsize=(10, 4))
env = gym.make("CliffWalking-v1", render_mode="rgb_array")
env.reset(seed=SEED)
axes[0].imshow(env.render())
axes[0].set_title("CliffWalking environment")
axes[0].axis("off")
env.close()

axes[1].imshow(np.zeros((4, 12)), cmap="Blues", vmin=0, vmax=1)
for row in range(4):
    for col in range(12):
        label = cliff[row, col] if cliff[row, col] in "SCG" else symbols[policy[row, col]]
        axes[1].text(col, row, label, ha="center", va="center", fontsize=20)
axes[1].set(xticks=range(12), yticks=range(4), xticklabels=[], yticklabels=[], titl
axes[1].grid(color="white", linewidth=2)
plt.tight_layout()
plt.show()
print("Policy action numbers (0=up, 1=right, 2=down, 3=left):")
print(policy)
```



Policy action numbers (0=up, 1=right, 2=down, 3=left):

```
[[1 1 1 1 1 1 1 1 1 1 1 2]
 [1 1 1 1 1 1 1 1 1 1 1 2]
 [1 1 1 1 1 1 1 1 1 1 1 2]
 [0 0 0 0 0 0 0 0 0 0 0 0]]
```

```
In [6]: # Minimal self-check for the tabular experiment.
assert set(summary_df["Training episodes"]) == set(EPISEODE_CONFIGS)
assert all(len(result["rewards"]) == episodes for episodes, result in results.items)
assert summary_df["Test success rate (%)"].between(0, 100).all()
print("Lab checks passed.")
```

Lab checks passed.

Lab 1 — Part B: Fuzzy Q-Learning for CartPole-v1

The original Part A above uses `CliffWalking-v1`. This section keeps that work unchanged and implements the requested fuzzy Q-learning experiments on `CartPole-v1`. Each continuous state is converted to a tuple of fuzzy linguistic labels, which indexes a Q-table.

Tasks covered: 2 and 5 fuzzy sets, triangular and trapezoidal membership functions, epsilon-greedy action selection, and comparisons for learning rates `0.01`, `0.05`, `0.1` and discount factors `0.5`, `0.9`. Increase `EPISODES` for a smoother learning curve.

```
In [7]: import itertools

import gymnasium as gym
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

SEED = 42
EPISODES = 500
MAX_STEPS = 500
FUZZY_SET_COUNTS = (2, 5)
MEMBERSHIP_TYPES = ("triangular", "trapezoidal")
LEARNING_RATES = (0.01, 0.05, 0.1)
DISCOUNTS = (0.5, 0.9)

# Practical clipping ranges for CartPole's unbounded velocity observations.
STATE_RANGES = np.array([[ -4.8, 4.8], [ -3.0, 3.0],
                          [ -0.418, 0.418], [ -4.0, 4.0]])

print(f"{len(FUZZY_SET_COUNTS) * len(MEMBERSHIP_TYPES) * len(LEARNING_RATES) * len(DISCOUNTS)}
```

24 configurations

```
In [8]: def triangular(x, a, b, c):
        return max(0.0, min((x - a) / (b - a), (c - x) / (c - b))) if a < b < c else float(0)

def trapezoidal(x, a, b, c, d):
    left = 1.0 if x >= b else (x - a) / (b - a) if b > a else 1.0
    right = 1.0 if x <= c else (d - x) / (d - c) if d > c else 1.0
    return max(0.0, min(left, right))

def membership_values(x, low, high, count, kind):
    x = float(np.clip(x, low, high))
    points = np.linspace(low, high, count)
    if count == 2:
        if kind == "triangular":
            return np.array([(high - x) / (high - low), (x - low) / (high - low)])
        mid = (low + high) / 2
        return np.array([trapezoidal(x, low, low, mid, high),
                          trapezoidal(x, low, mid, high, high)])
    values = []
    for i, center in enumerate(points):
```

```

        left = points[max(0, i - 1)]
        right = points[min(count - 1, i + 1)]
        if kind == "triangular":
            values.append(triangular(x, left, center, right))
        else:
            values.append(trapezoidal(x, left, center, center if i in (0, count - 1
return np.asarray(values)

def fuzzy_state(observation, set_count, kind):
    labels = [np.argmax(membership_values(value, low, high, set_count, kind))
               for value, (low, high) in zip(observation, STATE_RANGES)]
    return tuple(labels)

def state_index(state, set_count):
    return np.ravel_multi_index(state, (set_count,) * 4)

assert len(fuzzy_state([0, 0, 0, 0], 2, "triangular")) == 4
assert state_index((0, 0, 0, 0), 5) == 0

```

```

In [9]: def train_fuzzy_q(episodes=EPISODES, set_count=5, membership="triangular",
                        alpha=0.05, gamma=0.9, seed=SEED):
    env = gym.make("CartPole-v1")
    env.action_space.seed(seed)
    q_table = np.zeros((set_count ** 4, env.action_space.n))
    rng = np.random.default_rng(seed)
    rewards = []

    for episode in range(episodes):
        observation, _ = env.reset(seed=seed + episode)
        epsilon = max(0.05, 1.0 - 0.95 * episode / max(1, episodes - 1))
        total_reward = 0.0
        for _ in range(MAX_STEPS):
            state = state_index(fuzzy_state(observation, set_count, membership), se
            action = rng.integers(env.action_space.n) if rng.random() < epsilon els
            next_observation, reward, terminated, truncated, _ = env.step(action)
            next_state = state_index(fuzzy_state(next_observation, set_count, membe
            target = reward if terminated or truncated else reward + gamma * np.max
            q_table[state, action] += alpha * (target - q_table[state, action])
            observation = next_observation
            total_reward += reward
            if terminated or truncated:
                break
            rewards.append(total_reward)
        env.close()
    return q_table, np.asarray(rewards)

def evaluate(q_table, set_count, membership, episodes=50, seed=10_000):
    env = gym.make("CartPole-v1")
    scores = []
    for episode in range(episodes):
        observation, _ = env.reset(seed=seed + episode)
        score = 0

```

```

    for _ in range(MAX_STEPS):
        state = state_index(fuzzy_state(observation, set_count, membership), se
        observation, reward, terminated, truncated, _ = env.step(int(np.argmax(
        score += reward
        if terminated or truncated:
            break
        scores.append(score)
    env.close()
    return float(np.mean(scores))

```

```

In [10]: experiments = []
curves = {}
for set_count, membership, alpha, gamma in itertools.product(
    FUZZY_SET_COUNTS, MEMBERSHIP_TYPES, LEARNING_RATES, DISCOUNTS):
    q_table, rewards = train_fuzzy_q(set_count=set_count, membership=membership,
                                    alpha=alpha, gamma=gamma)
    key = (set_count, membership, alpha, gamma)
    curves[key] = rewards
    experiments.append({
        "Fuzzy sets": set_count, "Membership": membership,
        "Learning rate": alpha, "Discount factor": gamma,
        "Mean training reward (last 50)": rewards[-50:].mean(),
        "Greedy evaluation reward": evaluate(q_table, set_count, membership),
    })

summary_b = pd.DataFrame(experiments).sort_values("Greedy evaluation reward", ascen
display(summary_b)

```

	Fuzzy sets	Membership	Learning rate	Discount factor	Mean training reward (last 50)	Greedy evaluation reward
5	2	triangular	0.10	0.9	58.44	52.50
11	2	trapezoidal	0.10	0.9	58.44	52.50
13	5	triangular	0.01	0.9	42.28	46.88
17	5	triangular	0.10	0.9	37.84	45.76
15	5	triangular	0.05	0.9	36.66	45.52
4	2	triangular	0.10	0.5	45.60	44.02
10	2	trapezoidal	0.10	0.5	45.60	44.02
2	2	triangular	0.05	0.5	39.28	39.00
8	2	trapezoidal	0.05	0.5	39.28	39.00
14	5	triangular	0.05	0.5	27.82	36.64
3	2	triangular	0.05	0.9	22.86	29.24
9	2	trapezoidal	0.05	0.9	22.86	29.24
16	5	triangular	0.10	0.5	24.00	24.42
6	2	trapezoidal	0.01	0.5	16.60	14.82
0	2	triangular	0.01	0.5	16.60	14.82
12	5	triangular	0.01	0.5	12.14	11.16
21	5	trapezoidal	0.05	0.9	10.06	9.50
22	5	trapezoidal	0.10	0.5	9.64	9.50
18	5	trapezoidal	0.01	0.5	9.80	9.50
19	5	trapezoidal	0.01	0.9	9.78	9.50
7	2	trapezoidal	0.01	0.9	9.62	9.26
1	2	triangular	0.01	0.9	9.62	9.26
20	5	trapezoidal	0.05	0.5	10.30	9.26
23	5	trapezoidal	0.10	0.9	9.98	9.26

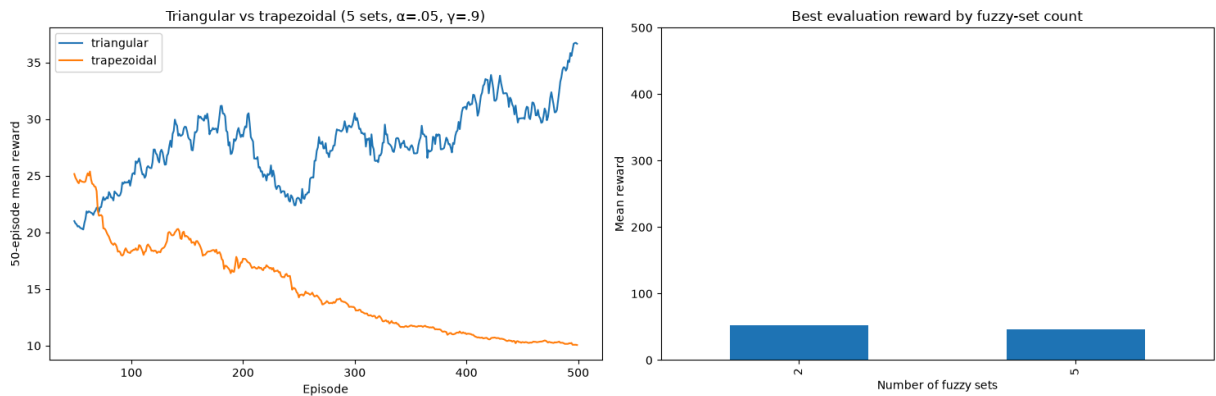
```
In [11]: fig, axes = plt.subplots(1, 2, figsize=(15, 5))
for membership in MEMBERSHIP_TYPES:
    key = (5, membership, 0.05, 0.9)
    axes[0].plot(pd.Series(curves[key]).rolling(50).mean(), label=membership)
axes[0].set(title="Triangular vs trapezoidal (5 sets, α=.05, γ=.9)",
            xlabel="Episode", ylabel="50-episode mean reward")
axes[0].legend()
```



```

best_by_sets = summary_b.groupby("Fuzzy sets")["Greedy evaluation reward"].max()
best_by_sets.plot(kind="bar", ax=axes[1], title="Best evaluation reward by fuzzy-se
                    ylabel="Mean reward", xlabel="Number of fuzzy sets", ylim=(0, 500
plt.tight_layout()
plt.show()

```



```

In [12]: # Minimal self-check for the fuzzy experiment.
assert set(summary_b["Fuzzy sets"]) == set(FUZZY_SET_COUNTS)
assert set(summary_b["Membership"]) == set(MEMBERSHIP_TYPES)
assert len(summary_b) == len(FUZZY_SET_COUNTS) * len(MEMBERSHIP_TYPES) * len(LEARNI
assert summary_b["Greedy evaluation reward"].between(0, MAX_STEPS).all()
print("Fuzzy CartPole checks passed.")

```

Fuzzy CartPole checks passed.